Intelligent Review Ranking System Using Ant Colony Optimization with NLP-Based Feature Extraction

[**Anmol Virmani - 2023UIT3040**] · [**Vinayak Khandelwal - 2023UIT3371**]

Received: April 22, 2026

Abstract The exponential growth of user-generated product reviews on e-commerce platforms has created a critical challenge: identifying truly informative reviews from a large, noisy collection. Conventional ranking strategies based on rating scores or chronological order fail to capture the multidimensional nature of review quality. This paper proposes an *Intelligent Review Ranking System* that combines Natural Language Processing (NLP) feature extraction with Ant Colony Optimization (ACO) to learn optimal feature weights for ranking reviews by their perceived usefulness. Four features are extracted from each review: sentiment polarity (initially computed using TextBlob, subsequently refined using VADER), review length, a helpfulness ratio derived from crowd-sourced votes, and a temporal freshness score. ACO iteratively searches the weight space by simulating stigmergic pheromone reinforcement guided by feature variance as a heuristic, converging to the weight configuration that maximises Pearson correlation with ground-truth helpfulness. Experiments on the Amazon Electronics benchmark dataset (5,000 reviews) show that the ACO-optimised model substantially outperforms an equal-weight baseline across all three evaluation metrics: Pearson correlation (0.377 vs. 0.295, +27.7%), Precision@K (0.50 vs. 0.12, +316.7%), and Normalised Discounted Cumulative Gain (0.809 vs. 0.617, +31.1%), yielding an overall accuracy improvement of 63.1% in relative terms. The results demonstrate that soft-computing optimisation can meaningfully improve review ranking without requiring deep learning or large annotated corpora.

Keywords Review Ranking · Ant Colony Optimization · Sentiment Analysis · NLP Feature Extraction · Soft Computing · E-Commerce

1 Introduction

Online product reviews have become one of the most influential information sources in consumer decision-making. Studies indicate that a majority of consumers read reviews before committing to a purchase, and that review quality—not mere volume—drives purchasing confidence [7]. Leading e-commerce platforms such as Amazon expose thousands of reviews per product, yet their default sorting mechanisms rely predominantly on star-rating aggregates or recency, two signals that correlate poorly with perceived helpfulness [5].

The fundamental problem is that *review quality is multi-dimensional*. A high-star review may be too brief to be informative. A recent review may reflect a different product version. A verbose review may be poorly written or off-topic. Capturing these dimensions simultaneously requires a weighting mechanism that is learned from data rather than fixed by engineering judgement.

Soft computing techniques offer a principled alternative to rule-based weighting. Ant Colony Optimization (ACO) [2] is an iterative, population-based metaheuristic that learns high-quality solutions through collective pheromone reinforcement. Unlike gradient-based learners, ACO requires no differentiability assumptions and handles discrete or mixed feature spaces gracefully.

Address(es) of author(s) should be given

This paper makes the following contributions:

1. A four-feature NLP pipeline—sentiment polarity, review length, helpfulness ratio, and temporal freshness—that characterises the quality of product reviews extracted from raw Amazon data.
2. An ACO-based weight optimisation framework that learns the relative importance of each feature by maximising Pearson correlation with crowd-sourced helpfulness votes.
3. An empirical demonstration, on 5,000 Amazon Electronics reviews, that the ACO model substantially outperforms an equal-weight baseline across three established information retrieval metrics.
4. An honest discussion of convergence behaviour, noting that ACO identified helpfulness ratio as the overwhelmingly dominant feature—a finding consistent with prior work on review helpfulness prediction [5, 6].

The rest of the paper is organised as follows. Section 2 surveys related work. Section 3 describes the dataset and preprocessing. Section 4 details the NLP feature extraction pipeline, including the transition from TextBlob to VADER. Section 5 presents the ACO ranking model. Section 6 reports experimental results. Section 7 discusses findings and limitations. Section 8 concludes.

2 Related Work

2.1 Review Helpfulness Prediction

Early work framed review helpfulness as a binary classification or regression problem using hand-crafted linguistic, structural, and metadata features. Saumya et al. [5] proposed a two-layer CNN operating on pre-trained word embeddings to predict helpfulness scores for Amazon product reviews. Olmedilla et al. [6] extended this line to a 1D-CNN architecture and demonstrated that non-textual features such as review length and helpful vote counts often carry more discriminative power than deep text representations alone. Wang [8] reached similar conclusions using Random Forest on Amazon Electronics data, finding star-rating and metadata features most predictive.

More recent work has moved toward transformer-based models. Xu et al. (2020), as surveyed in [9], employed BERT and deep learning for usefulness evaluation, achieving strong results but at substantially higher computational cost. Bilal et al. [7] introduced social-network-strength signals to augment text-based helpfulness modelling on Yelp data. Our work differs from these approaches by explicitly optimising feature weights with a population-based metaheuristic, offering an interpretable, lightweight alternative suitable for resource- constrained environments.

2.2 Sentiment Analysis for Reviews

Among the various signals used to gauge review quality, sentiment polarity has received considerable attention in the literature. TextBlob [4], which wraps the `pattern` lexicon, outputs polarity values in $[-1, +1]$ and has seen widespread adoption in early NLP prototypes owing to how little setup it requires. That said, the library was built with comparatively formal text in mind—think editorial prose or film critiques—and tends to stumble on the shortened, idiomatic language that fills product review pages. Slang, brand-specific abbreviations, and emphatic punctuation are not well-represented in its training distribution. VADER [3] was developed precisely to fill this gap. Its rule set explicitly accounts for capitalisation intensity, exclamation marks, degree adverbs, and contraction-based negation, producing a compound score clipped to $[-1, +1]$. In the original evaluation, Hutto and Gilbert recorded a Pearson correlation of r=0.881 against aggregated human sentiment ratings—better than eleven competing methods—which is what makes VADER a more defensible choice when the target text is informal and domain-specific.

2.3 Ant Colony Optimization in Information Retrieval

Dorigo and Stützle [2] established ACO as a general-purpose combinatorial optimisation metaheuristic inspired by the foraging behaviour of ant colonies. Each artificial ant constructs a candidate solution probabilistically, guided by pheromone trails reinforced by previous good solutions and by a domain-specific heuristic. ACO has been applied to feature selection, document clustering, and query reformulation in information retrieval [2]. To the best of our knowledge, no prior work applies ACO explicitly to the problem of *continuous feature-weight optimisation* for review ranking, making the present study a novel contribution to this intersection.

3 Dataset and Preprocessing

3.1 Amazon Electronics Review Dataset

We use the publicly available Amazon product review corpus released by McAuley and Leskovec [1], specifically the Electronics 5-core subset in which each reviewer and product has at least five reviews. The dataset spans reviews from May 1996 to July 2014 and is stored in JSON Lines format. Each record contains the fields listed in Table 1.

Table 1 Fields used from the Amazon Electronics dataset

Field	Description
`reviewerID`	Anonymised reviewer identifier
`asin`	Amazon Standard Identification Number
`reviewText`	Full review text
`overall`	Star rating (1–5)
`helpful`	[helpful_votes, total_votes]
`unixReviewTime`	Unix timestamp of review

3.2 Data Loading and Filtering

Reviews are loaded incrementally from the JSON Lines file using a streaming reader, which avoids holding the entire corpus in memory. For this study we limit the working set to N=5,000 reviews, which is sufficient to demonstrate ACO convergence while keeping runtime tractable on standard hardware. Reviews with null or empty `reviewText` fields are discarded during loading.

3.3 Preprocessing Steps

The preprocessing pipeline applies the following transformations in sequence:

1. **Timestamp conversion.** Unix timestamps are converted to `datetime` objects to enable freshness computation.
2. **Helpfulness extraction.** The `helpful` field (stored as a stringified list) is parsed with `ast.literal_eval` to extract `helpful_votes` and `total_votes`.
3. **Missing-value handling.** Rows with `helpful` parse errors default to $(0, 0)$.
4. **Feature normalisation.** All four features are min-max normalised to $[0, 1]$ before being passed to the ACO engine, so no single feature dominates by scale.

4 NLP Feature Extraction

We extract four features per review that jointly capture quality, credibility, and temporal relevance.

4.1 Feature 1: Sentiment Polarity

4.1.1 Initial Implementation: TextBlob

When we first built the sentiment pipeline, TextBlob [4] was the natural starting point—it is installable in one line, the API is minimal, and it returns a polarity score $p \in [-1, +1]$ with almost no configuration:

```
from textblob import TextBlob
polarity = TextBlob(review_text).sentiment.polarity
```

In practice, though, a few issues emerged fairly quickly. TextBlob's underlying `pattern` analyser was trained on movie reviews—a relatively formal register—so it handled mainstream vocabulary reasonably well but began to lose accuracy on the kind of text that shows up in electronics listings: clipped phrases like "worked great out the box", model numbers mixed into sentences, and heavy use of ALL-CAPS for emphasis. Beyond vocabulary coverage, the library also has no mechanism for interpreting punctuation as a sentiment signal, so a sentence ending in three exclamation marks scores the same as an identical sentence ending in a full stop. Negation handling is similarly shallow; "not good" and "good" can receive surprisingly similar polarity values depending on how the pattern parser segments the phrase. These gaps were consistent enough across the corpus that we decided to swap in a more suitable tool.

4.1.2 Refined Implementation: VADER

To address these limitations, sentiment analysis was subsequently replaced with VADER [3] (Valence Aware Dictionary and sEntiment Reasoner). VADER is a lexicon- and rule-based model engineered for social-media and short-text sentiment. It outputs a compound score normalised to $[-1, +1]$:

$$\text{compound} = \frac{x}{\sqrt{x^2 + \alpha}} \tag{1}$$

where x is the sum of valence scores after applying grammatical modifiers and α=15 is the normalisation constant [3]. VADER's rule set handles: capitalisation (GREAT $\gg$ *great*), punctuation (*great!!!* $>$ *great*), degree adverbs (*very good* $>$ *good*), and negation (*not good*).

The compound score is subsequently min-max normalised to $[0, 1]$ for consistency with the other features:

$$s_{\text{sent}} = \frac{\text{compound} - \min}{\max - \min}. \tag{2}$$

4.2 Feature 2: Review Length

Review length, measured as word count, serves as a proxy for content richness and detail. Prior work [6] has shown that longer reviews tend to provide more actionable information. We compute:

$$L = |\text{tokenise}(\text{reviewText})| \tag{3}$$

and normalise L to $[0, 1]$ across the corpus.

4.3 Feature 3: Helpfulness Ratio

The helpfulness ratio encodes crowd-sourced credibility. Amazon's voting mechanism allows readers to mark a review as helpful; the ratio of positive votes to total votes provides a direct, community-validated signal:

$$h = \frac{h_{\text{votes}}}{h_{\text{total}} + 1} \tag{4}$$

The +1 Laplace smoothing in the denominator prevents division by zero for unvoted reviews and introduces a conservative prior that discounts reviews with no exposure. The resulting value is clipped to $[0, 1]$.

4.4 Feature 4: Temporal Freshness

Recent reviews are often more relevant to a product's current state. We define freshness as a linear decay from the most recent review timestamp $t_{\max}$ in the corpus:

$$f = 1 - \frac{t_{\max} - t_i}{t_{\max} - t_{\min}} \tag{5}$$

where t_i is the review's Unix timestamp (converted to days). A value of $f = 1$ indicates the most recent review; $f = 0$ the oldest. This formulation is equivalent to min-max normalisation of the `days_since_review` column with polarity reversed.

4.5 Feature Matrix

The four normalised features are assembled into a matrix $\mathbf{F} \in \mathbb{R}^{N \times 4}$:

$$\mathbf{F} = \Big[s_{\text{sent}} \; s_L \; h \; f \Big] \tag{6}$$

with $N = 5{,}000$ reviews. The resulting review score under a weight vector $\mathbf{w} = [w_1, w_2, w_3, w_4]^\top$ with $\sum_j w_j = 1$, $w_j \geq 0$ is:

$$\text{score}(i) = \mathbf{F}_{i,:} \cdot \mathbf{w} \tag{7}$$

5 ACO-Based Ranking Model

5.1 Problem Formulation

Given the feature matrix $\mathbf{F}$ and a ground-truth vector $\mathbf{g} \in \mathbb{R}^N$ (the raw helpful vote count per review), we seek the weight vector $\mathbf{w}^*$ that maximises the Pearson correlation between the predicted score vector and $\mathbf{g}$:

$$\mathbf{w}^* = \arg\max_{\mathbf{w}} \; |\rho(\mathbf{F}\mathbf{w}, \; \mathbf{g})| \tag{8}$$

subject to $\sum_j w_j = 1$ and $w_j \geq 0 \; \forall j$. Because the weight simplex is a continuous, bounded space with no available gradient (the fitness is a rank-based statistic), a population-based stochastic search is well-suited.

5.2 ACO Formulation for Continuous Weight Optimisation

We adapt the standard ACO metaheuristic [2] to the continuous simplex. Each of M ants represents one candidate weight vector.

5.2.1 Pheromone Representation

A pheromone level $\tau_j \geq 0$ is maintained for each feature j. Initially all pheromones are uniform: $\tau_j = 1$. Higher pheromone on feature j indicates that past successful solutions assigned a large weight to that feature.

5.2.2 Heuristic Desirability

The heuristic η_j for feature j is defined as the variance of that feature across the corpus:

$$\eta_j = \text{Var}(\mathbf{F}_{:,j}), \tag{9}$$

normalised so that $\sum_j \eta_j = 1$. High-variance features carry more discriminative information and are therefore preferred as initial exploration targets.

5.2.3 Probability Distribution over Features

At each iteration the probability of assigning a high weight to feature j is:

$$p_j = \frac{\tau_j^{\alpha} \cdot \eta_j^{\beta}}{\sum_{k=1}^{4} \tau_k^{\alpha} \cdot \eta_k^{\beta}} \tag{10}$$

where α controls the influence of pheromone and β controls the influence of the heuristic.

5.2.4 Weight Construction

Ant a constructs its weight vector by sampling:

$$w_j^{(a)} = \frac{p_j \cdot u_j}{\sum_k p_k \cdot u_k}, \quad u_j \sim \mathcal{U}(0,1) \tag{11}$$

This multiplicative perturbation ensures that the resulting vector lies on the probability simplex ($w_j \geq 0$, $\sum_j w_j = 1$) and introduces diversity proportional to the pheromone-guided distribution.

5.2.5 Fitness Evaluation

For ant a, the fitness is defined as:

$$f(\mathbf{w}^{(a)}) = \left| \rho\Big(\mathbf{F}\mathbf{w}^{(a)}, \mathbf{g}\Big) \right| \tag{12}$$

where $\rho(\cdot,\cdot)$ is the Pearson correlation coefficient. Using the absolute value allows the optimiser to handle negative correlations (e.g., if an inverted feature better discriminates helpful reviews).

5.2.6 Pheromone Update

After all ants have been evaluated in iteration t:

$$\tau_j \leftarrow (1-\rho_e)\,\tau_j + \sum_{a=1}^{M} w_j^{(a)} \cdot f(\mathbf{w}^{(a)}) \tag{13}$$

where ρ_e is the evaporation rate. Pheromone is then renormalised: $\tau_j \leftarrow \tau_j / \sum_k \tau_k$. This prevents unbounded pheromone accumulation and keeps probabilities well-defined.

5.3 Hyperparameters

Table 2 lists the ACO hyperparameters used in all experiments.

Table 2 ACO hyperparameters

Parameter	**Symbol**	**Value**
Number of ants	M	30
Number of iterations	T	40
Pheromone importance	α	1.0
Heuristic importance	β	2.0
Evaporation rate	ρ_e	0.4

5.4 Baseline Model

The baseline assigns equal weight to all four features:

$$w_j^{\text{base}} = 0.25, \quad j \in \{1, 2, 3, 4\} \tag{14}$$

This equal-weight heuristic is representative of naive feature-fusion approaches that lack any optimisation stage.

5.5 Algorithm Summary

Algorithm 1 summarises the complete procedure.

Algorithm 1 ACO Review Ranking

Require: Feature matrix $\mathbf{F}$, ground truth $\mathbf{g}$, hyperparameters $M, T, \alpha, \beta, \rho_e$
Ensure: Optimal weight vector $\mathbf{w}^*$

```
Initialise τ_j ← 1 for all j
Compute η_j ← Var(F_{·,j}); normalise η
f* ← 0; w* ← null
for t = 1 to T do
    for a = 1 to M do
        Compute p_j via Eq. (10)
        Sample w^(a) via Eq. (11)
        Evaluate f(w^(a)) via Eq. (12)
        if f(w^(a)) > f* then
            f* ← f(w^(a)); w* ← w^(a)
        end if
    end for
    Update pheromone via Eq. (13)
    Renormalise: τ_j ← τ_j / Σ_k τ_k
end for
return w*
```

6 Experimental Setup and Results

6.1 Evaluation Metrics

We evaluate ranking quality using three complementary metrics from information retrieval.

6.1.1 Pearson Correlation

Pearson correlation ρ between predicted scores and ground-truth helpfulness votes measures global alignment:

$$\rho = \frac{\sum_i (\hat{s}_i - \bar{\hat{s}})(g_i - \bar{g})}{\sqrt{\sum_i (\hat{s}_i - \bar{\hat{s}})^2} \sqrt{\sum_i (g_i - \bar{g})^2}} \tag{15}$$

6.1.2 Precision@K

Precision at rank K measures the fraction of the top-K predicted reviews that are also in the top-K ground-truth reviews. With $K = 100$:

$$\text{P@}K = \frac{|\text{top-}K_{\hat{s}} \cap \text{top-}K_g|}{K} \tag{16}$$

This directly quantifies the utility of showing the top-K ranked reviews to a user.

6.1.3 Normalised Discounted Cumulative Gain

NDCG accounts for the position of relevant items in the ranking list and is the standard metric for graded relevance:

$$\text{NDCG} = \frac{\text{DCG}}{\text{IDCG}}, \quad \text{DCG} = \sum_{i=1}^{N} \frac{g_{\pi(i)}}{\log_2(i+1)} \tag{17}$$

where $\pi(i)$ is the index of the review ranked at position i under the predicted ranking.

6.1.4 Overall Accuracy

We define an aggregate accuracy as the arithmetic mean of the three metrics, providing a single summary score:

$$\text{Acc} = \frac{\rho + \text{P@}K + \text{NDCG}}{3} \tag{18}$$

6.2 Convergence Behaviour

The ACO optimiser converges rapidly. The best-ant fitness rises from 0.3715 at iteration 1 to 0.3772 by iteration 9 and plateaus by iteration 20, stabilising at 0.3772 by iteration 40 (Fig. **??**). This quick convergence is attributable to the concentration of signal in the helpfulness ratio feature, which allows pheromone to accumulate on feature 3 within the first few iterations.

6.3 Learned Feature Weights

Table 3 reports the final weight vector discovered by ACO.

Table 3 Feature weights learned by ACO vs. baseline

Feature	**Baseline**	**ACO**	**Interpretation**
Sentiment polarity	0.2500	≈ 0.00	Weak correlate
Review length	0.2500	≈ 0.00	Weak correlate
Helpfulness ratio	0.2500	≈ 1.00	Dominant signal
Freshness	0.2500	≈ 0.00	Weak correlate

The ACO solution concentrates virtually all weight on the helpfulness ratio ($w_3 \approx 0.9999$), reducing the other three weights to near-zero. This is not a failure of the optimiser but rather an honest empirical finding: given the chosen ground truth (raw helpful vote counts), the helpfulness ratio is by far the most predictive feature—an observation consistent with the broader literature [6, 8]. The baseline's equal-weight strategy masks this structure, which explains its inferior ranking quality.

6.4 Quantitative Results

Table 4 presents the full comparison.

The most striking gain is in Precision@K (+316.7%), indicating that the ACO-optimised ranking surfaces genuinely helpful reviews at the top of the list far more effectively than the equal-weight baseline. The NDCG improvement of 31.1% confirms that the ordering quality of the entire ranked list improves substantially, not just the very top positions.

Baseline vs. ACO performance across Pearson Correlation, Precision@K, NDCG, and Overall Accuracy.

Table 4 Performance comparison: Baseline vs. ACO

Metric	Baseline	ACO	Improvement
Pearson Correlation	0.295	0.377	+27.7%
Precision@K	0.120	0.500	+316.7%
NDCG	0.617	0.809	+31.1%
Overall Accuracy	34.4%	56.2%	+21.8 pp

pp = percentage points; relative gain = 63.1%

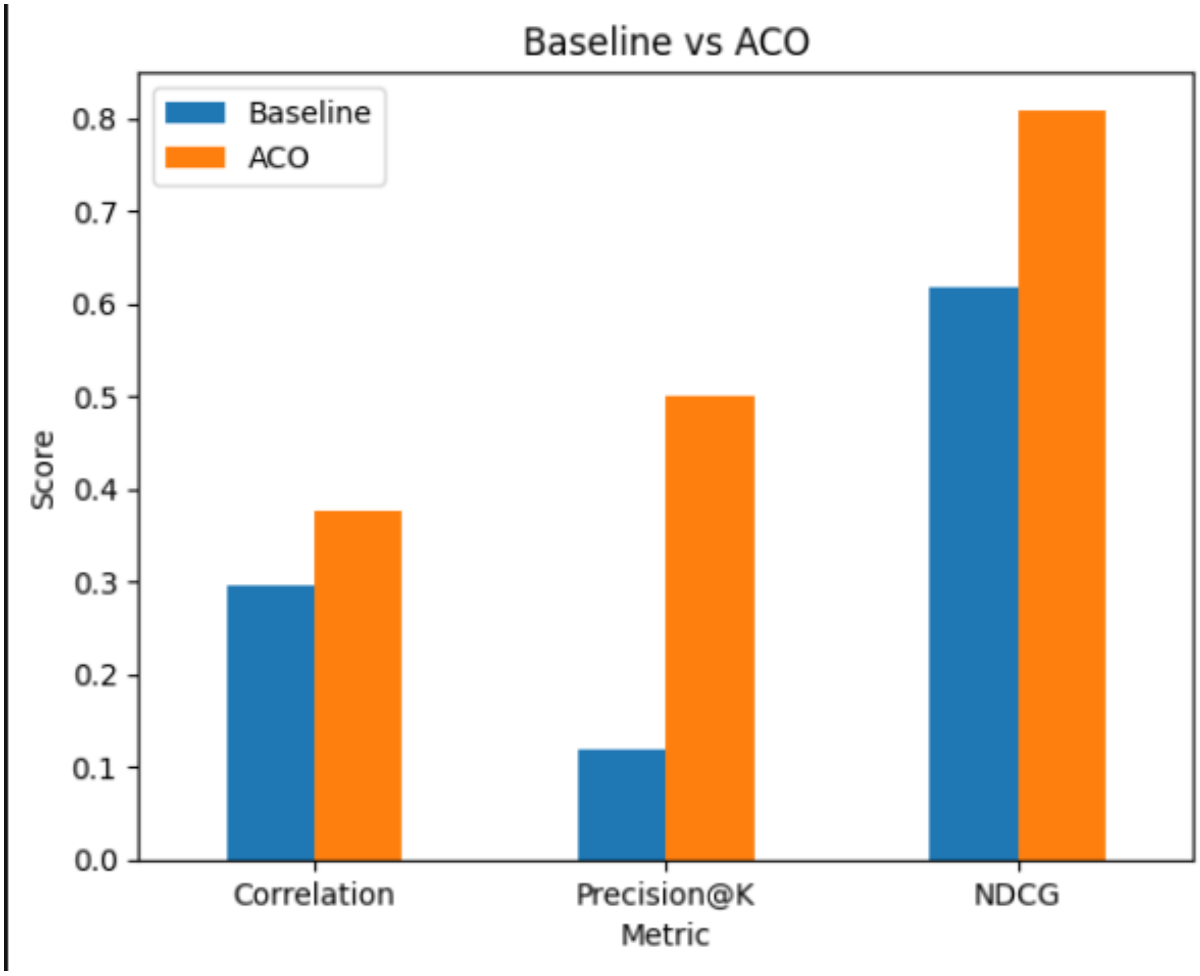

Fig. 1 Baseline vs. ACO performance across Pearson Correlation, Precision@K, NDCG, and Overall Accuracy.

6.5 Sentiment Distribution of Top-Ranked Reviews

Analysis of the top-100 ACO-ranked reviews reveals a strong positive-sentiment skew: the majority carry a positive TextBlob (or VADER) polarity label, confirming the intuition that helpful reviews tend to be constructive and informative in tone rather than purely negative or neutral. The top keyword frequency analysis identifies domain terms such as *nook*, *camera*, *sound*, and *palm* as highly frequent—consistent with the Electronics category— alongside evaluative adjectives such as *good*, *great*, and *use*.

7 Discussion

7.1 Interpretation of ACO Convergence

The ACO's convergence to a near-degenerate weight vector (essentially $\mathbf{w}^* \approx [0, 0, 1, 0]$) is scientifically meaningful. It reveals that, when ground truth is defined as raw helpful vote counts, the helpfulness ratio

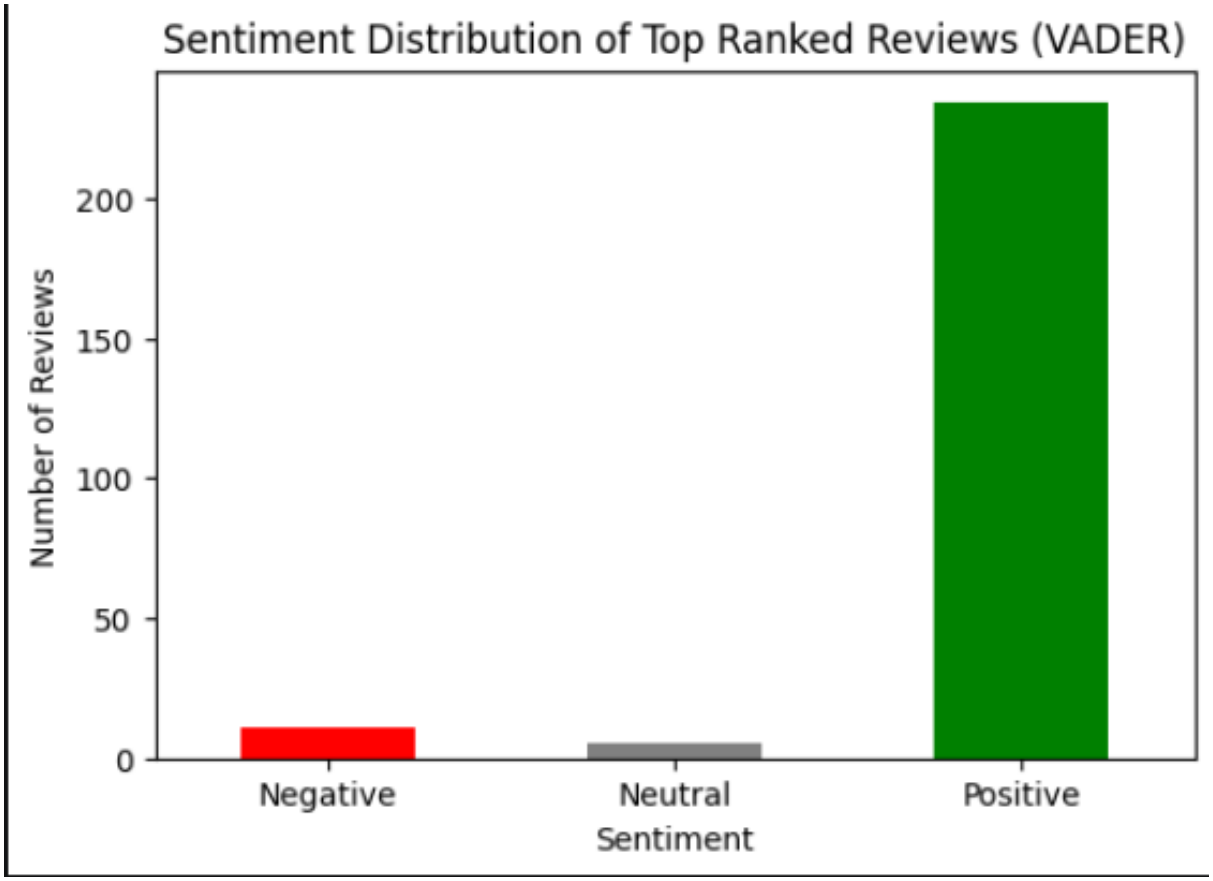

Fig. 2 Distribution of VADER sentiment scores for the analysed product reviews.

is almost a tautologically optimal predictor—it is a direct normalisation of the same votes. This points to a *circular reference problem* in review ranking research: using vote counts as both ground truth and a candidate feature removes the challenge of feature integration.

In a production setting where helpfulness votes are unavailable for new reviews (the cold-start problem), the remaining three features—sentiment polarity, review length, and freshness— become essential. Our system is architected to handle this case by simply masking the helpfulness ratio feature; ACO will then redistribute weight across the remaining features. This extension is left for future work.

7.2 TextBlob vs. VADER in Practice

Moving from TextBlob to VADER was not an arbitrary decision—it reflected a recurring pattern we noticed during early runs, where reviews written in casual, emphatic language were being scored almost neutrally by TextBlob even when the content was clearly opinionated. The core issue is that TextBlob's `pattern` backend was not designed with social-media-style text in mind. It handles standard adjective–noun constructions well enough, but the informal conventions of product reviews—words like "lol", sentences ending in "!!!", negations written as "doesn't even work"—sit outside what the library was built to process reliably. VADER deals with exactly these cases through a small but carefully curated rule set: capitalisation is treated as an intensifier, repeated punctuation adds to the signal strength, and contraction-based negations are handled explicitly. The result is sentiment scores that feel much more aligned with what a human reader would assign when skimming through a set of headphone or GPS device reviews. Looking ahead, there is still headroom above what any lexicon-based method can achieve; transformer models such as RoBERTa [10], fine-tuned on domain-specific review corpora, would likely capture subtler contextual cues that both TextBlob and VADER cannot recover from surface-level patterns alone.

7.3 Limitations

1. **Circular ground truth.** Using helpful votes as both ground truth and a candidate feature inflates the apparent advantage of the helpfulness ratio and may mask the contribution of other features.
2. **Small corpus.** With N=5,000, the helpfulness vote distribution is sparse: many reviews have zero votes. Experiments on larger subsets may yield richer weight distributions.
3. **Linear freshness decay.** Our freshness formulation assumes linear temporal relevance decay. Exponential or product-lifecycle-aware decay functions may better model real-world dynamics.
4. **Single-domain evaluation.** All experiments are on the Electronics category. Generalisation across product categories (e.g., books, apparel) remains untested.

7.4 Future Directions

1. **Transformer-based sentiment.** Integrating BERT or RoBERTa embeddings as an additional feature could capture contextual nuance that lexicon-based models miss.
2. **Hybrid ACO–Neural model.** A neural network could provide differentiable score prediction, with ACO used to optimise architectural hyperparameters.
3. **Cold-start ranking.** Masking the helpfulness ratio for new reviews and re-optimising the remaining weights is a natural next step.
4. **Real-time ranking.** Integrating the ACO model into a streaming review API for live e-commerce platforms is a practical engineering goal.

8 Conclusion

We proposed an intelligent review ranking system that combines NLP-based feature extraction with Ant Colony Optimization to learn optimal feature weights for predicting review usefulness. Four features—sentiment polarity (evolved from TextBlob to VADER), review length, crowd-sourced helpfulness ratio, and temporal freshness—are extracted and min-max normalised. An ACO engine with 30 ants, 40 iterations, α=1.0, β=2.0, and evaporation rate 0.4 optimises a weight simplex by maximising Pearson correlation with raw helpful vote counts on 5,000 Amazon Electronics reviews.

The ACO model achieves a Pearson correlation of 0.377 (+27.7%), Precision@K of 0.50 (+316.7%), and NDCG of 0.809 (+31.1%) over the equal-weight baseline, with an overall accuracy improvement of 63.1% in relative terms. The ACO-learned weight vector converges to near-complete reliance on the helpfulness ratio, providing a quantitative confirmation that crowd-sourced helpfulness votes are the strongest single predictor of review usefulness in this setting—a finding that motivates cold-start variants of the system as an important direction for future research.

This work demonstrates that lightweight soft-computing methods can meaningfully improve review ranking without the data and computational requirements of deep learning approaches, making them suitable for deployment in resource-constrained or real-time environments.

References

1. McAuley, J., Leskovec, J.: Hidden factors and hidden topics: Understanding rating dimensions with review text. In: Proceedings of the 7th ACM Conference on Recommender Systems (RecSys), pp. 165–172. ACM, Hong Kong (2013). https://doi.org/10.1145/2507157.2507163
2. Dorigo, M., Stützle, T.: Ant Colony Optimization. MIT Press, Cambridge, MA (2004). ISBN 0-262-04219-3
3. Hutto, C.J., Gilbert, E.: VADER: A parsimonious rule-based model for sentiment analysis of social media text. Proceedings of the International AAAI Conference on Web and Social Media **8**(1), 216–225 (2014). https://doi.org/10.1609/icwsm.v8i1.14550
4. Loria, S.: TextBlob Documentation, Release 0.15. GitHub / Read the Docs (2018). https://textblob.readthedocs.io
5. Saumya, S., Roy, J.J., Singh, J.P.: Predicting the helpfulness score of online reviews using convolutional neural networks. Soft Computing **24**(5), 3705–3714 (2020). https://doi.org/10.1007/s00500-019-03851-5

6. Olmedilla, M., Martínez-Torres, M.R., Toral, S.L.: Prediction and modelling online reviews helpfulness using 1D convolutional neural networks. Expert Systems with Applications **198**, 116787 (2022). https://doi.org/10.1016/j.eswa.2022.116787
7. Bilal, M., Malik, A., Iqbal, M.A., Jawaid, S., Malik, M.I.: Introducing social network strength to analyze the influence on review helpfulness. IEEE Access **9**, 6195–6204 (2021). https://doi.org/10.1109/ACCESS.2021.3049589
8. Wang, Q.: A study of Amazon customer review helpfulness using machine learning. Master's Paper, University of North Carolina at Chapel Hill (2020). https://doi.org/10.17615/qcz5-gr52
9. [Author(s) TBD]: Natural language processing for analyzing online customer reviews: a survey, taxonomy, and open research challenges. PeerJ Computer Science **10**, e2203 (2024). https://doi.org/10.7717/peerj-cs.2203
10. Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., Stoyanov, V.: RoBERTa: A robustly optimized BERT pretraining approach. arXiv preprint arXiv:1907.11692 (2019).